

Task 17: Reusing Logic with Custom Hooks

Summary

In this task, I learned how to create and use Custom Hooks in React to reuse stateful logic across multiple components. A Custom Hook is a JavaScript function whose name starts with `use` and can internally use other React Hooks such as `useState()` and `useEffect()`. This allows developers to extract common logic from components without duplicating code.

I also learned that Custom Hooks help improve code organization, readability, and maintainability by separating reusable logic from the UI. Instead of copying the same state management or Effect logic into different components, the logic can be written once inside a Custom Hook and reused wherever needed. This makes applications easier to maintain and reduces code duplication.

Additionally, I understood that Custom Hooks only share logic, not state. Every component that uses a Custom Hook gets its own independent state, even though they use the same Hook. This approach encourages reusable, modular, and scalable React applications while keeping components clean and focused on rendering the user interface.

Learning Outcome

After completing this task, I understood how to create and use Custom Hooks to reuse stateful logic across components, improve code reusability, and keep React applications clean, modular, and easier to maintain.

Task 18: Styling in React (CSS Modules, Inline Styles, and Styled Components)

Summary

In this task, I learned different ways to style React applications and organize CSS effectively. React supports multiple styling approaches, including external CSS files, CSS Modules, inline styles, and CSS-in-JS libraries such as Styled Components. Each method is useful depending on the size and requirements of the project.

I learned that CSS Modules provide locally scoped class names, preventing style conflicts between components and making large applications easier to maintain. I also understood how inline styles allow styles to be applied directly using JavaScript objects, which is useful for dynamic styling based on component state or props. Additionally, I explored how Styled Components let developers write CSS inside JavaScript, creating reusable styled components with encapsulated styles.

This task also helped me understand how to choose the appropriate styling method based on the project's needs. By using reusable styling techniques and component-based CSS, React applications become more organized, maintainable, and easier to scale as they grow.

Learning Outcome

After completing this task, I understood different styling techniques in React, including CSS Modules, inline styles, and Styled Components. I also learned how to organize component styles effectively and choose the most suitable styling approach for building clean, reusable, and scalable React applications.

⇒ GITHUB REPO LINK:

https://github.com/yashrathod910/15D_Internship/tree/main/DAY11